

: Hinglish Explanation

Is tutorial ka main purpose sirf ek searchable product table banana nahi hai. Ye actually sikhata hai ki **React application design karte waqt kis tarah sochna chahiye**.

React mein UI build karne ka overall process:

1. UI ko chhote **components** mein break karo.
2. Pehle data se ek static interface banao.
3. Identify karo ki kaunsa data **state** hona chahiye.
4. Decide karo ki state kis component mein rahegi.
5. User interactions ke through state update karwao.

Example application mein products ko category-wise dikhaya gaya hai. User product search kar sakta hai aur checkbox se sirf in-stock products dekh sakta hai.

Starting Point: Data aur UI Mockup

Humein ek API se products ka data mil raha hai:

```
1  [  
2    {  
3      category: "Fruits",  
4      price: "$1",  
5      stocked: true,  
6      name: "Apple"  
7    }  
8  ]
```

Har product ke paas chaar properties hain:

- category: Product kis category mein aata hai
- price: Product ki price
- stocked: Product stock mein hai ya nahi
- name: Product ka naam

React application banane se pehle humein do cheezein samajhni hain:

- Data ki structure kya hai?
- UI mein us data ko kaise represent kiya gaya hai?

Agar data properly structured hai, to aksar data ki structure aur component hierarchy ek-dusre se naturally match karti hain.

Step 1: UI ko Component Hierarchy mein Break Karo

Sabse pehle mockup ke alag-alag sections ke around imaginary boxes draw karo. Har independent section ko ek component ki tarah dekho.

Is application mein paanch components hain:

```
1 FilterableProductTable
2   |— SearchBar
3   |— ProductTable
4     |— ProductCategoryRow
5     |— ProductRow
```

1. FilterableProductTable

Ye poori application ka top-level component hai.

Iske andar:

- Search interface
- Product table

dono present hain.

2. SearchBar

Ye user input handle karne wala section hai.

Isme:

- Search input
- “Only show products in stock” checkbox

present hai.

3. ProductTable

Ye products ki filtered list table ke form mein display karta hai.

Ye search text aur checkbox value ke basis par decide karega ki kaunse products show karne hain.

4. ProductCategoryRow

Ye har category ka heading render karta hai:

- | | |
|---|------------|
| 1 | Fruits |
| 2 | Vegetables |

5. ProductRow

Ye ek individual product ki row render karta hai:

- | | | |
|---|---------|-----|
| 1 | Apple | \$1 |
| 2 | Spinach | \$2 |

Component Banane ka Decision Kaise Lein?

Har HTML element ko component banana zaroori nahi hai.

Naya component tab useful hota hai jab:

- Uska ek clear responsibility ho
- Wo multiple jagah reuse ho sakta ho
- Existing component bahut bada ho gaya ho
- Us section ki apni logic ho
- Us section ko independently understand karna useful ho

Tutorial mein table header ko separate component nahi banaya gaya:

```
1 <thead>
2   <tr>
3     <th>Name</th>
4     <th>Price</th>
5   </tr>
6 </thead>
```

Kyunki abhi header simple hai aur sirf ProductTable ke context mein use ho raha hai.

Lekin agar future mein sorting add ho:

- Sort by name
- Sort by price
- Ascending/descending controls

tab ise ProductTableHeader component mein move karna sensible ho sakta hai.

Main principle hai **separation of concerns**, yani ideally ek component ko ek main responsibility handle karni chahiye.

Step 2: Pehle Static Version Banao

Component hierarchy decide karne ke baad pehle application ka **static version** build karo.

Static version ka matlab:

- Data render hoga
- Components create honge
- Props pass honggi
- Lekin search aur checkbox abhi kaam nahi karenge
- State use nahi hogi

Is approach ka benefit ye hai ki display logic aur interaction logic ko ek saath solve nahi karna padta.

Pehle UI banao, phir usmein behavior add karo.

ProductCategoryRow

```
1 function ProductCategoryRow({ category }) {  
2   return (  
3     <tr>  
4       <th colspan="2">  
5         {category}  
6       </th>  
7     </tr>  
8   );  
9 }
```

Ye component category ko prop ke through receive karta hai:

```
1 <ProductCategoryRow category="Fruits" />
```

colspan="2" ka matlab category heading table ke dono columns cover karegi:

1		Fruits	
2		Apple	\$1

ProductRow

```
1 function ProductRow({ product }) {  
2   const name = product.stocked  
3     ? product.name  
4     : <span style={{ color: 'red' }}>  
5       {product.name}
```

```

6      </span>;
7
8      return (
9        <tr>
10         <td>{name}</td>
11         <td>{product.price}</td>
12       </tr>
13     );
14   }

```

Ye component ek product object receive karta hai.

Agar product stock mein hai:

```

1  product.stocked === true

```

to normal naam show hota hai.

Agar product stock mein nahi hai:

```

1  product.stocked === false

```

to naam red color mein show hota hai.

Ye conditional rendering hai:

```

1  condition ? valueIfTrue : valueIfFalse

```

Example:

```

1  const name = product.stocked
2    ? product.name
3    : <span style={{ color: 'red' }}>{product.name}</span>;

```

ProductTable

ProductTable ka kaam products array ko table rows mein convert karna hai:

```

1  function ProductTable({ products }) {
2    const rows = [];
3    let lastCategory = null;
4
5    products.forEach((product) => {
6      // Rows banana
7    });
8
9    return (
10     <table>

```

```
11      ...
12      <tbody>{rows}</tbody>
13    </table>
14  );
15 }
```

rows array

```
1  const rows = [];
```

Is array mein JSX elements store kiye ja rahe hain:

```
1  <ProductCategoryRow />
2  <ProductRow />
3  <ProductRow />
```

React JSX elements ko arrays ke andar render kar sakta hai.

lastCategory

```
1  let lastCategory = null;
```

Is variable ka use track karne ke liye ho raha hai ki previous product kis category ka tha.

Products data category-wise ordered hai:

```
1  Fruits
2  Fruits
3  Fruits
4  Vegetables
5  Vegetables
6  Vegetables
```

Jab current product ki category previous category se different hoti hai, tab ek naya category heading add kiya jata hai:

```
1  if (product.category !== lastCategory) {
2    rows.push(
3      <ProductCategoryRow
4        category={product.category}
5        key={product.category}
6      />
7    );
8  }
```

Uske baad har product ke liye ProductRow add hota hai:

```
1  rows.push(
2    <ProductRow
3      product={product}
```

```
4     key={product.name}
5   />
6 );
```

Finally, current category ko save kar diya jata hai:

```
1 lastCategory = product.category;
```

key Prop Kyun Di Gayi Hai?

Jab React mein list ke through components render hote hain, har item ko ek unique key deni padti hai:

```
1 <ProductRow
2   product={product}
3   key={product.name}
4 />
```

Yahan assumption hai ki har product ka name unique hai.

Category heading ke liye category name key hai:

```
1 <ProductCategoryRow
2   category={product.category}
3   key={product.category}
4 />
```

SearchBar ka Static Version

```
1 function SearchBar() {
2   return (
3     <form>
4       <input
5         type="text"
6         placeholder="Search..."
7       />
8
9       <label>
10        <input type="checkbox" />
11        Only show products in stock
12      </label>
13    </form>
14  );
15 }
```

Abhi input aur checkbox visible hain, lekin ye kisi state se connected nahi hain. Isliye ye product table ko control nahi karte.

One-Way Data Flow

Top-level component products ko prop ke through receive karta hai:

```
1 function FilterableProductTable({ products }) {  
2   return (  
3     <div>  
4       <SearchBar />  
5       <ProductTable products={products} />  
6     </div>  
7   );  
8 }
```

Data ka flow:

```
1 App  
2 ↓  
3 FilterableProductTable  
4 ↓  
5 ProductTable  
6 ↓  
7 ProductRow
```

Data parent se child ki taraf ja raha hai. React mein ise **one-way data flow** kaha gaya hai.

```
1 export default function App() {  
2   return (  
3     <FilterableProductTable products={PRODUCTS} />  
4   );  
5 }
```

Yahan PRODUCTS, App se FilterableProductTable ko diya gaya. Phir FilterableProductTable ne wahi data ProductTable ko pass kar diya.

Static version mein state use nahi karni chahiye, kyunki abhi data user interaction ke result mein change nahi ho raha.

Step 3: Minimal State Identify Karo

Application ko interactive banane ke liye React ko kuch changing information remember karni hogi. Ye information **state** mein rakhi jati hai.

Main principle:

State mein sirf minimum necessary changing data store karo.

Jo value already props ya existing state se calculate ho sakti hai, use alag state mein store nahi karna chahiye.

Is Application ka Possible Data

Application mein chaar important values hain:

1. Original products list
2. User ka search text
3. Checkbox ki value
4. Filtered products list

Ab decide karte hain ki inme se kaunsi value state hai.

State Check Karne ke Questions

Kisi value ke liye ye questions poochho:

1. Kya value time ke saath unchanged rehti hai?

Agar haan, wo state nahi hai.

2. Kya value parent component se prop ke through mil rahi hai?

Agar haan, wo is component ki state nahi hai.

3. Kya value existing props ya state se calculate ho sakti hai?

Agar haan, wo definitely alag state nahi honi chahiye.

Products List State Nahi Hai

Original products parent se props ke through mil rahe hain:

```
1 <FilterableProductTable products={PRODUCTS} />
```

Isliye products list ko duplicate karke state mein rakhne ki zaroorat nahi hai.

Search Text State Hai

Search text user type karta hai:

```
1 apple
2 pea
3 fruit
```

Ye time ke saath change hota hai aur kisi doosri value se calculate nahi ho sakta.

Isliye search text state hai:

```
1 const [filterText, setFilterText] = useState('');
```

Checkbox Value State Hai

Checkbox user check aur uncheck kar sakta hai.

Ye bhi change hota hai aur kisi doosri value se calculate nahi hota.

Isliye ye state hai:

```
1 const [inStockOnly, setInStockOnly] = useState(false);
```

Filtered Products State Nahi Hai

Filtered list in teen values se calculate ki ja sakti hai:

```
1 products + filterText + inStockOnly
```

Isliye filtered products ki separate state rakhne ki zaroorat nahi hai.

Agar separate state rakhenge, to duplicated data create hoga:

```
1 const [products, setProducts] = useState(...);
2 const [filteredProducts, setFilteredProducts] = useState(...);
```

Phir jab original products ya filters change honge, to filteredProducts ko synchronize rakhna padega. Isi unnecessary duplication ko avoid karna **DRY**, yani “Don’t Repeat Yourself”, principle ka part hai.

Final minimal state:

```
1 filterText
2 inStockOnly
```

Props vs State

Props

Props function arguments ki tarah hoti hain.

Parent component child ko data deta hai:

```
1 <ProductTable products={products} />
```

Child ise receive karta hai:

```
1 function ProductTable({ products }) {  
2   // products use karo  
3 }
```

Props ka purpose child ko customize karna aur parent se child tak data bhejna hai.

State

State component ki memory ki tarah hoti hai:

```
1 const [filterText, setFilterText] = useState('');
```

- filterText: Current stored value
- setFilterText: Value update karne ka function
- "": Initial value

State interaction ke response mein change ho sakti hai.

Props aur State Saath Kaise Kaam Karte Hain?

Parent apne andar state rakhta hai:

```
1 const [filterText, setFilterText] = useState('');
```

Phir us state ko child ko prop ke form mein pass karta hai:

```
1 <SearchBar filterText={filterText} />
```

Yani ek value parent ke liye state ho sakti hai aur child ke liye prop.

Step 4: State Kahan Rehni Chahiye?

State identify karne ke baad next question hai:

Is state ka owner kaunsa component hoga?

Har state ke liye:

1. Identify karo kaunse components us state ko use karenge.

2. Un components ka closest common parent find karo.
 3. State ko common parent ya uske upar rakho.
-

Search State Kaun Use Karta Hai?

SearchBar

SearchBar ko search text input mein display karna hai:

```
1 <input value={filterText} />
```

Checkbox ki current value bhi display karni hai:

```
1 <input  
2   type="checkbox"  
3   checked={inStockOnly}  
4 />
```

ProductTable

ProductTable ko search text aur checkbox ke basis par products filter karne hain.

Dono components state use kar rahe hain:

```
1 FilterableProductTable  
2 |— SearchBar  
3 |— ProductTable
```

Inka closest common parent hai:

```
1 FilterableProductTable
```

Isliye state FilterableProductTable mein rakhi jayegi.

```
1 function FilterableProductTable({ products }) {  
2   const [filterText, setFilterText] = useState('');  
3   const [inStockOnly, setInStockOnly] = useState(false);  
4  
5   // ...  
6 }
```

Is technique ko commonly state ko **lift up** karna kaha jata hai. Yani state ko individual child ke andar rakhne ke bajay unke common parent mein rakha jata hai.

State ko Props ke Through Pass Karna

State ko SearchBar ko pass kiya gaya:

```
1 <SearchBar
2   filterText={filterText}
3   inStockOnly={inStockOnly}
4 />
```

Aur wahi state ProductTable ko bhi pass ki gayi:

```
1 <ProductTable
2   products={products}
3   filterText={filterText}
4   inStockOnly={inStockOnly}
5 />
```

Resulting flow:

```
1 FilterableProductTable state
2   |
3   ├── filterText → SearchBar
4   ├── inStockOnly → SearchBar
5   |
6   ├── filterText → ProductTable
7   └── inStockOnly → ProductTable
```

Ek hi source of truth hai: FilterableProductTable.

Product Filtering Logic

ProductTable har product ko check karta hai:

```
1 products.forEach((product) => {
2   // Filtering logic
3 });
```

Search Text Filter

```
1 if (
2   product.name.toLowerCase().indexOf(
3     filterText.toLowerCase()
4   ) === -1
5 ) {
6   return;
7 }
```

Dono strings lowercase ki jati hain:

```
1 product.name.toLowerCase()
```

```
2 filterText.toLowerCase()
```

Isse search case-insensitive ho jati hai.

Example:

```
1 Product: "Apple"
2 Search:  "app"
```

Check effectively ye banega:

```
1 "apple".indexOf("app")
```

Agar search text nahi milta, indexOf() -1 return karta hai. Us situation mein current product ko rows mein add nahi kiya jata.

In-Stock Filter

```
1 if (inStockOnly && !product.stocked) {
2   return;
3 }
```

Iska meaning hai:

- Checkbox checked hai
- Aur product stock mein nahi hai

To product skip kar do.

Agar inStockOnly false hai, to stock mein aur out-of-stock dono products show honge.

Controlled Inputs aur Read-Only Problem

Step 4 mein input ko state ke saath connect kar diya gaya:

```
1 <input
2   type="text"
3   value={filterText}
4 />
```

Checkbox bhi state ke saath connected hai:

```
1 <input
2   type="checkbox"
3   checked={inStockOnly}
4 />
```

Lekin ab user input edit nahi kar sakta.

Kyun?

Input ki value React state se fix ho rahi hai:

```
1 value={filterText}
```

Lekin user type karne par state update karne ke liye onChange exist nahi karta.

React basically keh raha hai:

Input ki value hamesha filterText ke equal rakho.

Agar filterText update hi nahi hoga, input ki displayed value bhi nahi badlegi. Isliye field read-only behave karegi.

Step 5: Inverse Data Flow Add Karo

Normal data flow parent se child hai:

```
1 FilterableProductTable
2     ↓
3     SearchBar
```

Lekin user interaction SearchBar ke andar hoti hai:

```
1 User types in SearchBar
```

State parent FilterableProductTable mein hai. Isliye child ko kisi tarah parent ki state update karni hogi.

React mein parent apne state-update functions child ko props ke through deta hai:

```
1 <SearchBar
2   filterText={filterText}
3   inStockOnly={inStockOnly}
4   onFilterTextChange={setFilterText}
5   onInStockOnlyChange={setInStockOnly}
6 />
```

Yahan:

```
1 setFilterText
```

ko child mein:

```
1 onFilterTextChange
```

naam ki prop ke through pass kiya gaya.

Similarly:

```
1 setInStockOnly
```

ko:

```
1 onInStockOnlyChange
```

prop banakar pass kiya gaya.

Search Input Update Karna

```
1 <input
2   type="text"
3   value={filterText}
4   placeholder="Search..."
5   onChange={(e) =>
6     onFilterTextChange(e.target.value)
7   }
8 />
```

User jab input mein type karta hai, onChange event run hota hai.

e event object hai:

```
1 e.target
```

actual input element ko represent karta hai.

```
1 e.target.value
```

input ka latest text deta hai.

Example:

```
1 User types: app
```

Then:

```
1 e.target.value === "app"
```

Ye value parent ke setter ko di jati hai:

```
1 onFilterTextChange("app");
```

Actually ye parent ka function hai:

```
1 setFilterText("app");
```

State update hote hi React component ko dobara render karta hai. Updated search text SearchBar aur ProductTable, dono ko props ke through mil jata hai.

Checkbox Update Karna

```
1 <input
2   type="checkbox"
3   checked={inStockOnly}
4   onChange={(e) =>
5     onInStockOnlyChange(e.target.checked)
6   }
7 />
```

Checkbox ke liye:

```
1 e.target.checked
```

use kiya jata hai, kyunki checkbox ki relevant value Boolean hoti hai:

```
1 true
2 false
```

Input text ke liye:

```
1 e.target.value
```

Checkbox ke liye:

```
1 e.target.checked
```

Complete Data Flow

Search input ka complete flow:

```
1 1. User input mein type karta hai
2 2. SearchBar ka onChange run hota hai
3 3. onFilterTextChanged latest text receive karta hai
4 4. Parent ka setFilterText call hota hai
5 5. filterText state update hoti hai
6 6. FilterableProductTable dobara render hota hai
7 7. New filterText ProductTable ko milta hai
8 8. ProductTable products ko dobara filter karta hai
9 9. Screen par filtered products dikhte hain
```

Checkbox ke liye:

```
1 1. User checkbox check karta hai
2 2. onChange event run hota hai
3 3. e.target.checked true deta hai
4 4. setInStockOnly(true) call hota hai
5 5. Parent state update hoti hai
```

- | | |
|---|--|
| 6 | 6. ProductTable dobara render hota hai |
| 7 | 7. Out-of-stock products skip ho jate hain |

“Inverse Data Flow” ka Actual Meaning

Iska matlab ye nahi ki React ka normal data direction reverse ho gaya.

State ab bhi parent mein hai aur state values ab bhi neeche props ke through flow karti hain:

- | | |
|---|-----------------------------|
| 1 | State value: Parent → Child |
|---|-----------------------------|

Lekin child user interaction report karne ke liye parent se mila callback call karta hai:

- | | |
|---|-------------------------------------|
| 1 | User event: Child → Parent callback |
|---|-------------------------------------|

Complete model:

1	FilterableProductTable
2	
3	— values down:
4	filterText
5	inStockOnly
6	
7	— callbacks down:
8	onFilterTextChange
9	onInStockOnlyChange
10	
11	SearchBar user event
12	
13	— callback call karke parent state update

Ye data flow explicit hai. Is wajah se clearly pata rehta hai:

- State kahan stored hai
- State ka owner kaun hai
- State kaun read kar raha hai
- State kis event se update ho rahi hai

Final Architecture

App

Data provide karta hai:

```
1 <FilterableProductTable products={PRODUCTS} />
```

FilterableProductTable

State own karta hai:

```
1 const [filterText, setFilterText] = useState('');  
2 const [inStockOnly, setInStockOnly] = useState(false);
```

Values aur callbacks children ko pass karta hai.

SearchBar

State values display karta hai aur user events par callbacks call karta hai.

ProductTable

Props ke basis par products filter karta hai aur table rows generate karta hai.

ProductCategoryRow

Category heading display karta hai.

ProductRow

Individual product ka name aur price display karta hai. Out-of-stock product ka naam red karta hai.

Is Tutorial se Kya Yaad Rakhna Hai?

1. UI ko components mein break karo

Har component ki clear responsibility honi chahiye.

```
1 SearchBar → search controls  
2 ProductTable → table aur filtering  
3 ProductRow → single product
```

2. Static version pehle banao

Pehle components, JSX aur props se UI render karo. Interactivity baad mein add karo.

3. Har changing value state nahi hoti

State sirf wo minimum changing data hai jise application ko remember karna zaroori hai.

Is example mein:

```
1 State:
```

```
2  ✓ filterText
3  ✓ inStockOnly
4
5  State nahi:
6  ✗ products, kyunki props se mil rahe hain
7  ✗ filtered products, kyunki calculate ho sakte hain
```

4. State ko closest common parent mein rakho

Agar do sibling components ko same state chahiye, state ko commonly unke parent mein rakho.

5. Props neeche flow karti hain

Parent child ko values deta hai:

```
1  <ProductTable filterText={filterText} />
```

6. Child callbacks se parent ko update request deta hai

```
1  onChange={(e) =>
2    onFilterTextChange(e.target.value)
3  }
```

7. Derived data ko calculate karo

Filtered products ko separate state mein store karne ke bajay render ke waqt products, search text aur checkbox value se calculate kiya gaya.

Short Mental Formula

React UI banate waqt ye sequence follow kar sakte ho:

```
1  Mockup dekho
2    ↓
3  Components identify karo
4    ↓
5  Component hierarchy banao
6    ↓
7  Static UI banao
8    ↓
9  Minimal state identify karo
10   ↓
```

```
11 State owner choose karo
12   ↓
13 State ko props se neeche bhejo
14   ↓
15 Callbacks se user interaction handle karo
```

Ek line mein:

React mein application ko components mein todte hain, minimum state common parent mein rakhte hain, values props se neeche bhejte hain, aur child events ko callback functions se parent state tak pahunchate hain.

Input mein diye gaye external links

- https://en.wikipedia.org/wiki/Separation_of_concerns
- https://en.wikipedia.org/wiki/Don%27t_repeat_yourself